

Data challenge : anomaly detection with diffusion models

Option IDSC, Mines Paris

September, 28 - October, 2, 2026

1 Introduction

Time-frequency representations of signals emitted in geophysics applications are dominated by a stochastic, spatially correlated noise. In the presence of an actual emitted signal, narrow, elongated structures appear as almost-straight segments in the time-frequency plane. These line-like patterns are events of interest and their automatic detection is the practical goal of this data challenge.

Two properties of the problem make it significantly different from a standard supervised detection or segmentation task. First, confirmed examples of lines are rare and costly to obtain: identifying and labeling them requires a human rater to inspect individual observations. Training a discriminative classifier or a segmentation network directly on “line” versus “no-line” examples would require a labeled dataset, possibly non-representative of future, unseen emission. Second, examples of pure background noise, free of any known emission, are essentially unlimited: they can be drawn automatically from any segment of the signal recorded when no emission is known to be present, with no manual annotation effort at all.

These two observations motivate framing the problem as *unsupervised anomaly detection*: rather than learning to recognize what a line looks like, one learns a generative model of what the background noise looks like, using only unlabeled, noise-only observations. At detection time, an observation is declared anomalous to the extent that it deviates from what this model considers typical. The generative model that we will consider is a *denoising diffusion model*.

The data challenge relies on a diffusion model that is already implemented: its code can be found in the folder `src/`, the two trained networks in the folder `models/`, and the folder `scripts/` contains an example showing how to run it. The equations and notations corresponding to the code are presented in Sections 2 and 3. The code does not need to be modified: only the last part of the detection pipeline, described in Section 5, is the subject of the data challenge.

2 Diffusion models

Diffusion models are built around a simple idea: it is far easier to learn how to gradually destroy structure in data than to learn how to create it from scratch. Starting from a real example, one repeatedly adds a small amount of random noise in many small steps, until nothing recognizable remains and the result is indistinguishable from pure static noise. This noising process is entirely mechanical and requires no learning, since exactly how much noise has been added at each step is known by construction. A neural network is then trained to undo this process one small step at a time: shown a noisy version of an example together with how far along the noising process it is, it learns to guess what noise was just added, and therefore how to nudge the sample slightly back towards a cleaner version. Because each individual step only has to undo a small amount of corruption, this is a much easier learning problem than trying to produce a

clean sample outright. Once trained, the network can be applied repeatedly, starting from pure noise and walking backward through the same sequence of steps, gradually turning randomness into a sample that looks as though it could have come from the training data.

We reuse the same mechanism to perform anomaly detection: corrupting an observation just enough to blur out small, atypical structure, then letting the trained network denoise it back, tends to erase whatever the network never learned to expect, while faithfully restoring whatever already looked normal.

2.1 Notations and main equations

This section introduces, in the order in which they are used, all the quantities that appear in the rest of the note.

Data. A clean observation is denoted $\mathbf{x}_0 \in \mathbb{R}^d$, where the subscript 0 means “not corrupted”. In this challenge, $\mathbf{x}_0$ is a normalized spectrogram patch of size $H \times W$ (Section 3.2), so that $d = H \times W$. Normal (anomaly-free) patches are assumed to be independent draws from an unknown probability distribution p_{data} , which is precisely what the diffusion model learns to represent.

Forward (noising) process. The noising process is indexed by an integer *timestep* $t \in \{0, 1, \dots, T\}$, where T is the total number of diffusion steps ($T = 1000$ in the reference configuration). The amount of noise injected at step t is set by a *variance schedule* $\beta_1, \dots, \beta_T \in (0, 1)$, fixed before training. We use a linear schedule, $\beta_t = \beta_{\min} + \frac{t-1}{T-1}(\beta_{\max} - \beta_{\min})$ with $\beta_{\min} = 10^{-4}$ and $\beta_{\max} = 2 \times 10^{-2}$ [1]. One step of the forward process shrinks the current sample slightly and adds a small Gaussian perturbation:

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \mathbf{z}_t, \quad \mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad t = 1, \dots, T, \quad (1)$$

where $\mathcal{N}(\mathbf{0}, \mathbf{I})$ denotes the standard Gaussian distribution on $\mathbb{R}^d$ (independent pixels with zero mean and unit variance). Two derived quantities are used everywhere:

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s, \quad \text{with the convention } \bar{\alpha}_0 = 1. \quad (2)$$

$\bar{\alpha}_t$ decreases from $\bar{\alpha}_0 = 1$ (no corruption) to $\bar{\alpha}_T \approx 0$ (pure noise). Composing the t Gaussian steps of Eq. (1) shows that $\mathbf{x}_t$ can be sampled directly from $\mathbf{x}_0$, without simulating the intermediate steps:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (3)$$

In words, $\mathbf{x}_t$ is a weighted mix of the clean patch and of a single Gaussian noise image $\boldsymbol{\epsilon}$, with a signal weight $\sqrt{\bar{\alpha}_t}$ and a noise weight $\sqrt{1 - \bar{\alpha}_t}$ whose squares sum to one. Since the patches have unit variance (Section 3.2), $\mathbf{x}_t$ keeps unit variance at every timestep. The ratio $\bar{\alpha}_t/(1 - \bar{\alpha}_t)$ is the *signal-to-noise ratio* at timestep t .

Noise-prediction network. The only learned component is a neural network $\boldsymbol{\epsilon}_\theta$, with trainable parameters θ . It takes as input a corrupted patch $\mathbf{x}_t$ and its timestep t , and outputs an image $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \in \mathbb{R}^d$ of the same size, which is its estimate of the noise $\boldsymbol{\epsilon}$ that was mixed into $\mathbf{x}_0$ in Eq. (3). The architecture is described in Section 3.3.

Training objective. Following [1], the network is trained to minimize the mean squared error between the true and the predicted noise,

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}_0 \sim p_{\text{data}}, t \sim \mathcal{U}\{1, \dots, T\}, \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right\|_2^2 \right], \quad \mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad (4)$$

where $\mathbb{E}$ denotes the expectation over the three random quantities listed in subscript, $\mathcal{U}\{1, \dots, T\}$ is the uniform distribution over the timesteps and $\|\cdot\|_2$ is the Euclidean norm over the d pixels. In practice, the expectation is approximated by an average over a mini-batch: each training patch receives its own random timestep and its own random noise image. No label is ever needed.

Predicted clean patch. Once the network is trained, inverting Eq. (3) with the predicted noise in place of the true one gives an estimate of the clean patch from any corrupted one:

$$\hat{\mathbf{x}}_0(\mathbf{x}_t, t) = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}}. \quad (5)$$

Score. The *score* of a probability density p is the gradient of its logarithm, $\nabla_{\mathbf{x}} \log p(\mathbf{x})$: a vector field that points, at every $\mathbf{x}$, in the direction in which the density increases fastest, i.e. towards more typical samples. Let p_t denote the distribution of $\mathbf{x}_t$ when $\mathbf{x}_0 \sim p_{\text{data}}$ (the data distribution “blurred” by the noise of level t). A classical result [4, 5, 6] shows that the noise-prediction network provides an estimate of the score of p_t :

$$\mathbf{s}_\theta(\mathbf{x}_t, t) = -\frac{\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{1 - \bar{\alpha}_t}} \approx \nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t). \quad (6)$$

Removing the predicted noise from $\mathbf{x}_t$ therefore amounts to moving $\mathbf{x}_t$ along $\mathbf{s}_\theta$, towards regions of high density under the (blurred) distribution of *normal* patches.

Deterministic reverse process (DDIM). To go back from a noise level t to a lower noise level $t' < t$, we use the deterministic update of DDIM [2]: the clean patch is first predicted with Eq. (5), then re-noised to level t' using the *predicted* noise instead of a fresh random one,

$$\mathbf{x}_{t'} = \sqrt{\bar{\alpha}_{t'}} \hat{\mathbf{x}}_0(\mathbf{x}_t, t) + \sqrt{1 - \bar{\alpha}_{t'}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t). \quad (7)$$

Because this update contains no random term, the same input always yields the same output. It also does not require $t' = t - 1$: the reverse process can jump along a short, decreasing subsequence of timesteps $\tau_0 > \tau_1 > \dots > \tau_K$, which reduces the number of network evaluations from T to K . When the last timestep is $\tau_K = 0$, the convention $\bar{\alpha}_0 = 1$ makes the last update return the predicted clean patch $\hat{\mathbf{x}}_0$.

2.2 Application to anomaly detection

Suppose that $\mathbf{x}_0$ is *not* a typical sample of p_{data} , but an observation that is close to the support of p_{data} everywhere except on a small, structured subset, like for instance, a normal-looking background texture with a thin, coherent line superimposed on it, as it is the case with the spectrogram images:

- first, because the score estimator $\mathbf{s}_\theta(\cdot, t)$ of Eq. (6) was fit exclusively on samples from p_{data} , it has learned a vector field that consistently points towards the support of p_{data} at every noise level, with no comparable “pull” towards the out-of-distribution structure.
- second, at a moderate noise level, the small-scale, low-probability structure is largely overwhelmed by the injected Gaussian perturbation, while the coarse statistics of the observation remain informative for the reverse process.

Consequently, integrating the reverse trajectory from $\mathbf{x}_{t_{\text{start}}}$ back to $t = 0$ tends to regenerate plausible, typical texture in place of the untypical structure, while reconstructing the typical part of the observation with high fidelity, since that is exactly what the model was trained to do.

3 Methodology

We present in this section the detailed methodology used in this work to detect emissions in spectrograms.

3.1 Data model and problem formalization

In what follows, we model the spectrogram representation of a signal as a real-valued field $s : \Omega \rightarrow \mathbb{R}$ over a discrete grid Ω of frequency and time bins. Under normal operating conditions, s is supposed to be a realization of a stochastic texture process with distribution P_0 : a spatially correlated but statistically homogeneous noise field, with no long-range coherent structure. Under anomalous conditions, the observed field is modeled as a superposition

$$s = s_0 + a, \quad s_0 \sim P_0, \quad (8)$$

where a is an additive component whose support is concentrated along an approximately linear curve in the time-frequency plane, and whose probability of occurrence under P_0 is, by construction, negligible.

The detection problem is posed locally: a bounded observation window, or *patch*, $\mathbf{x} \in \mathbb{R}^{H \times W}$ is extracted from s (Section 3.2), and the task is to decide, from $\mathbf{x}$ alone, whether it contains such a structured component and, if so, to characterize its orientation. The distribution p_{data} of Section 2.1 is the distribution of the (normalized) patches extracted from fields $s_0 \sim P_0$.

3.2 Patch representation and normalization

Local observation windows $\mathbf{x} \in \mathbb{R}^{H \times W}$, with $H = W = 48$, are extracted from the pre-processed representation on a regular grid, without overlap: two neighboring windows share a common boundary, so that a line-like structure crossing this boundary is split between two adjacent patches (Section 4 explains how to identify adjacent patches). The first axis of a patch (rows, of size H) is the frequency axis and the second axis (columns, of size W) is the time axis. Each extracted patch is independently standardized to zero mean and unit variance,

$$\tilde{\mathbf{x}} = \frac{\mathbf{x} - \mu_{\mathbf{x}}}{\sigma_{\mathbf{x}} + \varepsilon}, \quad (9)$$

where $\mu_{\mathbf{x}}$ and $\sigma_{\mathbf{x}}$ are the mean and standard deviation of the d pixel values of $\mathbf{x}$, and ε is a small constant (10^{-8} in the code) that prevents a division by zero. Patches whose standard deviation, prior to normalization, falls below a small threshold are discarded as uninformative, since they correspond to near-silent regions carrying no usable texture. This normalization is applied consistently throughout the pipeline, both when assembling the training set and when scoring a candidate patch at detection time, so that the network always operates on inputs of comparable dynamic range. Only patches drawn from regions with no known emission are used to fit the model of P_0 . In the rest of the note, $\mathbf{x}_0$ always denotes a normalized patch $\tilde{\mathbf{x}}$.

3.3 Noise-prediction network

We implement the noise-prediction network $\epsilon_{\theta}(\mathbf{x}_t, t)$ of Eq. (4) as a multi-scale convolutional encoder-decoder with skip connections between encoder and decoder stages at matching spatial resolutions (a U-Net). The corruption level t is conditioned on by first mapping it to a fixed-dimensional embedding through a sinusoidal encoding at geometrically spaced frequencies, analogous to positional encodings used in sequence models, followed by a small feed-forward projection. The resulting embedding is then injected

Table 1: The two trained networks provided for the challenge (files `models/unet32.pt` and `models/unet64.pt`).

	unet32	unet64
Base channel width	32	64
Number of parameters	2.2×10^6	8.2×10^6
Diffusion steps T	1000	1000
Noise schedule	linear, $\beta_t \in [10^{-4}, 2 \times 10^{-2}]$	
Optimizer, learning rate	Adam, 10^{-4}	Adam, 10^{-4}
Batch size	16	32
Optimization steps	200,000	100,000

into every convolutional block of the network as an additive, per-channel bias, so that a single set of convolutional weights is reused at every corruption level.

Each convolutional block consists of two convolutions followed by normalization and a non-linearity, with the conditioning bias added after the first normalization. The encoder halves the spatial resolution and doubles the channel width at each of several stages, a bottleneck block operates at the coarsest resolution and widest channel count, and the decoder mirrors the encoder, upsampling and concatenating the corresponding cached encoder activation (the skip connection) at each stage before applying a further convolutional block; a final linear projection maps back to a single output channel, interpreted as the predicted noise $\epsilon_\theta(\mathbf{x}_t, t)$.

The network depth (number of encoder/decoder stages), base channel width, and conditioning-embedding dimension are hyperparameters. In the two networks provided for the challenge (Table 1), the encoder has 3 stages, so that a 48×48 patch is reduced to 6×6 at the bottleneck, the embedding of t has dimension 256, and the two networks only differ by their base channel width.

3.4 Training procedure

The network is trained with the objective of Eq. (4): for every mini-batch of standardized, normal-only patches, an independent corruption level $t \sim \mathcal{U}\{1, \dots, T\}$ is drawn per sample, the corresponding corrupted patch $\mathbf{x}_t$ is formed via Eq. (3), and the network parameters θ are updated by stochastic gradient-based optimization (Adam) to minimize the mean squared error between the true and predicted noise.

Two networks, trained on about 400 000 normal patches with this procedure, are provided for the challenge: you do not have to train a diffusion model yourself. Their configurations are given in Table 1. They share the same noise schedule, so that a given timestep t corresponds to the same noise level $\sqrt{1 - \bar{\alpha}_t}$ for both of them. The smaller network, `unet32`, is about three times faster to evaluate than `unet64`, which matters when the whole test database must be processed on a CPU; comparing the two networks allows you to check whether your conclusions depend on the model.

3.5 Anomaly scoring by partial diffusion purification

At detection time, a candidate patch $\mathbf{x}_0$ is scored using the trained, frozen noise-prediction network through the following procedure (Algorithm 1):

1. **Partial forward corruption.** Rather than starting from pure Gaussian noise, which would discard all information about $\mathbf{x}_0$, the patch is forward-corrupted only up to a fixed, intermediate timestep $t_{\text{start}} < T$ via Eq. (3):

$$\mathbf{x}_{t_{\text{start}}} = \sqrt{\bar{\alpha}_{t_{\text{start}}}} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{t_{\text{start}}}} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

Algorithm 1 Anomaly scoring by partial diffusion

Require: normalized patch $\mathbf{x}_0$, frozen noise-prediction network ϵ_θ , schedule $\bar{\alpha}_1, \dots, \bar{\alpha}_T$ (with $\bar{\alpha}_0 = 1$), start timestep t_{start} , number of reverse steps K

- 1: $\tau_0, \dots, \tau_K \leftarrow$ decreasing timesteps from $\tau_0 = t_{\text{start}}$ to $\tau_K = 0$
- 2: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 3: $\mathbf{x}_{\tau_0} \leftarrow \sqrt{\bar{\alpha}_{\tau_0}} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{\tau_0}} \epsilon$ $\triangleright$ Eq. (3)
- 4: **for** $i = 0, \dots, K - 1$ **do**
- 5: $\hat{\epsilon} \leftarrow \epsilon_\theta(\mathbf{x}_{\tau_i}, \tau_i)$ $\triangleright$ predicted noise
- 6: $\hat{\mathbf{x}}_0 \leftarrow (\mathbf{x}_{\tau_i} - \sqrt{1 - \bar{\alpha}_{\tau_i}} \hat{\epsilon}) / \sqrt{\bar{\alpha}_{\tau_i}}$ $\triangleright$ Eq. (5)
- 7: $\mathbf{x}_{\tau_{i+1}} \leftarrow \sqrt{\bar{\alpha}_{\tau_{i+1}}} \hat{\mathbf{x}}_0 + \sqrt{1 - \bar{\alpha}_{\tau_{i+1}}} \hat{\epsilon}$ $\triangleright$ Eq. (7)
- 8: **end for**
- 9: $\hat{\mathbf{x}}_0 \leftarrow \mathbf{x}_{\tau_K}$ $\triangleright \bar{\alpha}_{\tau_K} = \bar{\alpha}_0 = 1$
- 10: $e \leftarrow |\mathbf{x}_0 - \hat{\mathbf{x}}_0| / (1 + |\hat{\mathbf{x}}_0|)$ $\triangleright$ Eq. (10)
- 11: **return** e

The corruption level is chosen large enough for the added Gaussian noise to overwhelm a thin, low-amplitude structured component, but small enough for the statistics of the patch to remain informative for the reverse process.

2. **Accelerated deterministic denoising.** A decreasing sequence of $K + 1$ timesteps $t_{\text{start}} = \tau_0 > \tau_1 > \dots > \tau_K = 0$, roughly evenly spaced, is constructed. The DDIM update of Eq. (7) is applied K times, from $\mathbf{x}_{\tau_i}$ to $\mathbf{x}_{\tau_{i+1}}$, and yields a reconstruction $\hat{\mathbf{x}}_0$, supposed **not to contain** the patterns of interest.
3. **Residual computation.** The final anomaly map is the pixelwise, normalized residual between the observation and its purified reconstruction,

$$e(\mathbf{x}_0, \hat{\mathbf{x}}_0) = \frac{|\mathbf{x}_0 - \hat{\mathbf{x}}_0|}{1 + |\hat{\mathbf{x}}_0|} \in [0, \infty), \quad (10)$$

where the absolute value, the division and the sum are taken pixel by pixel, so that e is itself an $H \times W$ image. Normalizing the absolute difference by $1 + |\hat{\mathbf{x}}_0|$ turns it into a *relative* rather than a raw absolute error: it discounts discrepancies at locations where the reconstruction itself has large magnitude, where a given absolute deviation is correspondingly less surprising, and it keeps the residual on a comparable scale across observations whose purified reconstructions differ in overall dynamic range. The constant 1 in the denominator avoids dividing by values of $\hat{\mathbf{x}}_0$ close to zero.

The corruption level t_{start} directly trades off anomaly removal against fidelity of the normal-texture reconstruction: a larger t_{start} destroys more of any structured anomaly before reconstruction, but a t_{start} that is too large also erases genuine fine detail from the normal texture, inflating the residual, and hence the false-positive rate, everywhere. Note also that, because of the random noise image ϵ drawn in the first step, two runs of Algorithm 1 on the same patch give slightly different residual maps, unless the random seed is fixed.

4 Test database

Content. The test database consists of 2863 patches of size 48×48 (frequency $\times$ time), stored as individual NumPy files (.npz, 32-bit floats) in the directory `dataset/test/`.

Normalization. The stored values are *not* standardized patch by patch: before being fed to the network, each test patch must be normalized with Eq. (9), exactly as the

Table 2: Composition of the annotated subset of the test database (502 patches).

Labels	Number of patches
No line (horizontal = other = false)	364
Horizontal line only	57
Other orientation only	68
Both	13
Total with horizontal = true	70
Total with other = true	81

training patches.

Annotations. A subset of 502 test patches has been inspected by a human rater and annotated. The annotations are stored in the JSON file `dataset/annotations.json`, which maps each annotated filename to two independent booleans:

- **horizontal**: the patch contains a horizontal line, that is a narrow structure at an approximately constant frequency, extending along the time axis;
- **other**: the patch contains a line-like structure with another orientation, that is an oblique line; vertical structures are not considered as lines and are not annotated.

Both labels can be true for the same patch. The additional field **checked**, present for some patches, is a bookkeeping field of the annotation process and can be ignored. The composition of the annotated subset is given in Table 2. Note that the classes are unbalanced: about 73% of the annotated patches contain no line at all.

5 Your task: from the residual map to a decision

The two networks of Table 1 give access to the whole purification procedure of Section 3.5. The objective of the data challenge is to build the last stage of the pipeline, which seeks, for each test patch, at turning the patch $\mathbf{x}_0$ and its residual map e into two decisions, **horizontal** and **other**, depending on whether or not horizontal lines or lines with other orientations are present (excluding vertical lines). Note that the method must remain *unsupervised*: the annotations cannot be used to train a classifier.

Possible angles of attack.

- **Purification parameters.** How do the corruption level t_{start} (equivalently, the noise level $\sqrt{1 - \bar{\alpha}_{t_{\text{start}}}}$), the number of reverse steps K and the choice of the network influence the residual maps? How much do the results vary with the random noise ϵ ?
- **The residual map itself.** Eq. (10) is one choice among many: signed difference $\mathbf{x}_0 - \hat{\mathbf{x}}_0$ (the lines are brighter than the background), average over several random seeds or several values of t_{start} , other normalizations, or the score $\mathbf{s}_\theta$ of Eq. (6) used as an anomaly map.
- **Line detection.** Oriented filter banks (e.g. Gabor filters), Radon or Hough transforms, or any other way of detecting elongated, oriented structures in e and of estimating their orientation.
- **Decision.** Aggregation of local evidence over the patch or over sub-regions, choice of the thresholds (for instance calibrated to a prescribed false-alarm rate on normal patches), and use of the patches that are adjacent in time (Section 4), since a line may continue from one patch to the next.
- **Lines of other orientations.** Oblique lines are harder to detect than horizontal

ones and may call for a specific treatment.

Evaluation. Choosing how to evaluate your detector is part of the data challenge. Using the 502 annotated patches, you will propose and justify an evaluation metric, taking into account the characteristics of the problem: two criteria, unbalanced classes, a small number of annotated patches, and the risk of an optimistic evaluation if the annotations are also used to tune your method.

References

- [1] J. Ho, A. Jain, and P. Abbeel. *Denoising Diffusion Probabilistic Models*. Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [2] J. Song, C. Meng, and S. Ermon. *Denoising Diffusion Implicit Models*. International Conference on Learning Representations (ICLR), 2021.
- [3] C. Meng, Y. He, Y. Song, J. Song, J. Wu, J.-Y. Zhu, and S. Ermon. *SDEdit: Guided Image Synthesis and Editing with Stochastic Differential Equations*. International Conference on Learning Representations (ICLR), 2022.
- [4] P. Vincent. *A Connection Between Score Matching and Denoising Autoencoders*. Neural Computation, 23(7), 2011.
- [5] Y. Song and S. Ermon. *Generative Modeling by Estimating Gradients of the Data Distribution*. Advances in Neural Information Processing Systems (NeurIPS), 2019.
- [6] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. *Score-Based Generative Modeling through Stochastic Differential Equations*. International Conference on Learning Representations (ICLR), 2021.